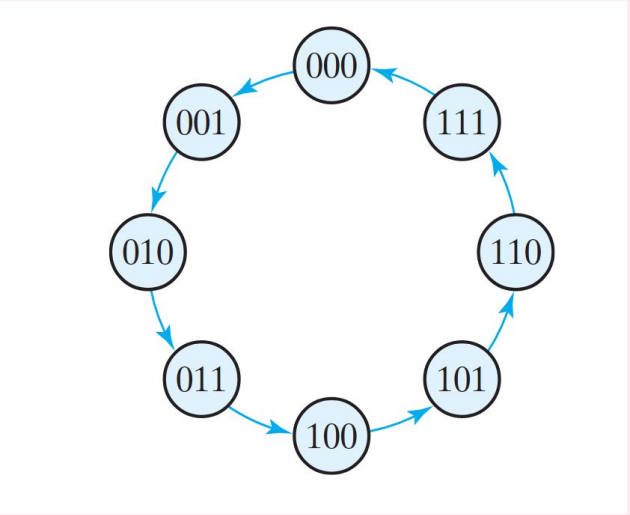
Synchronous and Asynchronous Counters

Vishesh Agrawal
Divyam Gupta

Counters

- Counters are machines which increment their value at a fixed interval
- Counters are extremely vital for any machine and are used extensively in digital electronics



Behavioral Modelling Counters in Verilog

- We can implement the counters very simply in verilog using behavioral modelling
- We define a register that stores our value and at every positive edge, we increment or decrement that value.

Code Examples

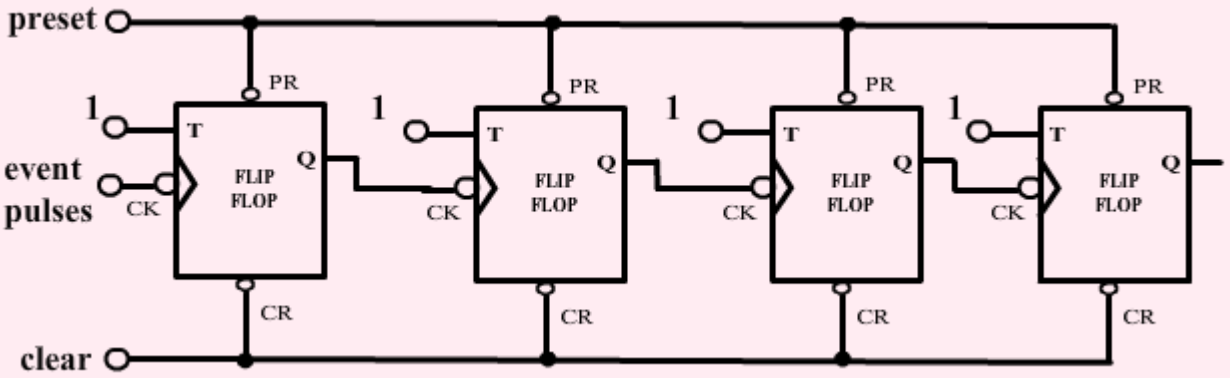
```
module down_counter (
    input clk,
    input reset,
    output reg[2:0] out
);
    always @(posedge clk) begin
        if (reset) begin
            out <= 3'b000;
        end
        else begin
            out <= out - 1;
        end
    end
endmodule
```

```
module up_counter (
    input clk,
    input reset,
    output reg[2:0] out
);
    always @(posedge clk) begin
        if (reset) begin
            out <= 3'b000;
        end
        else begin
            out <= out + 1;
        end
    end
endmodule
```

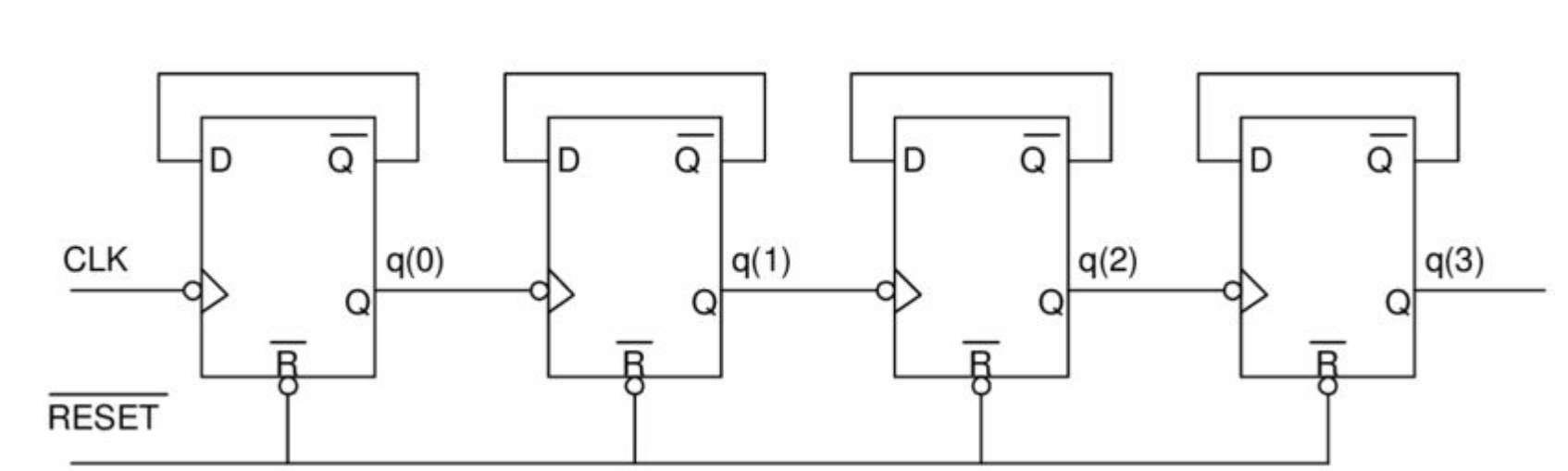
Asynchronous Counters

- Asynchronous counters use toggling using the clock as their counting mechanism
- Each flip flop uses the previous flip flops' output as its clock
- So because of this rippling effect they are also called ripple counters

Ripple Counter using T Flip Flop



Ripple Counter using D Flip Flops



4-bit Ripple Counter Verilog Implementation

```verilog
module t_ff (
    input  wire clk,
    input  wire reset,
    input  wire t,
    output reg  q
);
    always @(posedge clk or posedge reset) begin
        if (reset)
            q <= 1'b0;
        else if (t)
            q <= ~q;
    end
endmodule
```

```verilog
module ripple_counter_4bit (
    input  wire clk,
    input  wire reset,
    output wire [3:0] q
);

    wire q0, q1, q2, q3;

    t_ff ff0(.clk(clk),  .reset(reset), .t(1'b1), .q(q0));
    t_ff ff1(.clk(q0),   .reset(reset), .t(1'b1), .q(q1));
    t_ff ff2(.clk(q1),   .reset(reset), .t(1'b1), .q(q2));
    t_ff ff3(.clk(q2),   .reset(reset), .t(1'b1), .q(q3));

    assign q = {q3, q2, q1, q0};
endmodule

```

Synchronous Counters

- All flip-flops share the **same clock signal**.
- **Changes occur simultaneously** on each clock pulse
- **Faster and more accurate** than asynchronous counters.
- Used in **timers, frequency dividers, and digital clocks**.

.

Implementation of Synchronous Counter using T Flip Flop

Qn	Qn+1	T
0	0	0
0	1	1
1	0	1
1	1	0

Present State				Next State				Inputs of FF			
QD	QC	QB	QA	QD+1	QC+1	QB+1	QA+1	TD	TC	TB	TA
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	1	0	0	1	0	0	0	1	1
0	0	1	0	0	0	1	1	0	0	0	1
0	0	1	1	0	1	0	0	0	1	1	1
0	1	0	0	0	1	0	1	0	0	0	1
0	1	0	1	0	1	1	0	0	0	1	1
0	1	1	0	0	1	1	1	0	0	0	1
0	1	1	1	1	0	0	0	1	1	1	1
1	0	0	0	1	0	0	1	0	0	0	1
1	0	0	1	1	0	1	0	0	0	1	1
1	0	1	0	1	0	1	1	0	0	0	1
1	0	1	1	1	1	0	0	0	1	1	1
1	1	0	0	1	1	0	1	0	0	0	1
1	1	0	1	1	1	1	0	0	0	1	1
1	1	1	0	1	1	1	1	0	0	0	1
1	1	1	1	0	0	0	0	1	1	1	1

Flip Flop Derived Inputs from K-Map

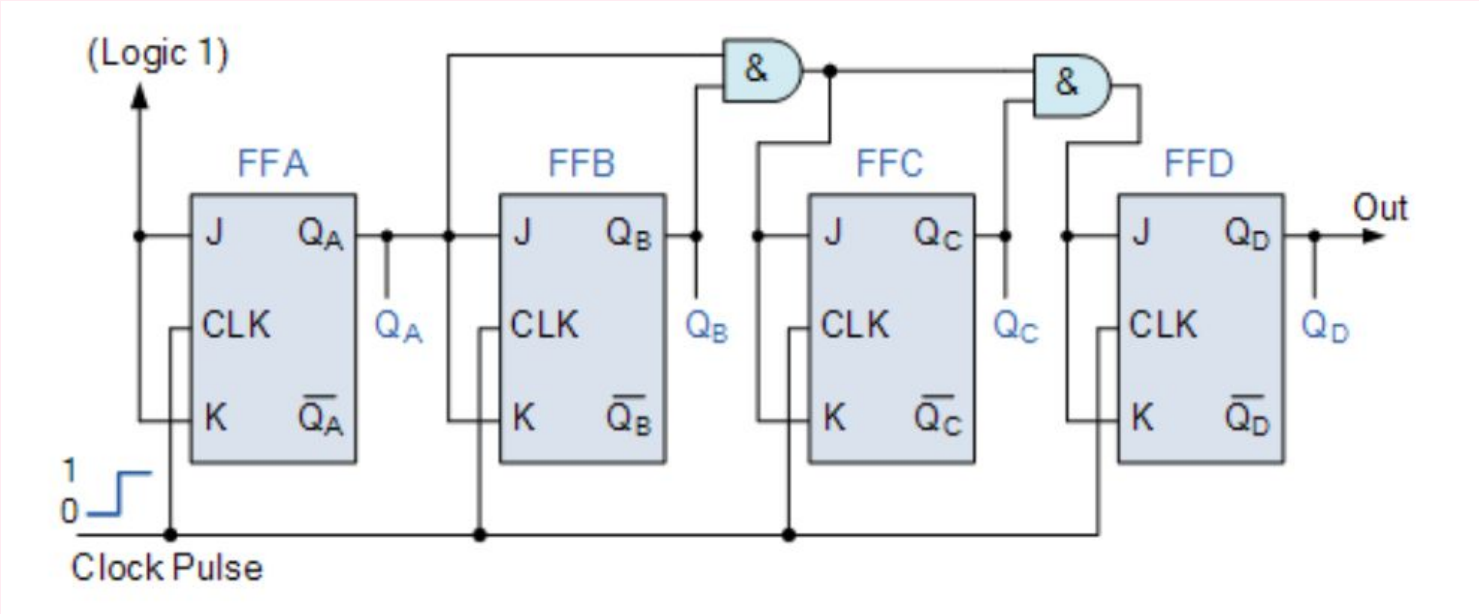
$$TA = 1$$

$$TB = Q_A$$

$$TC = Q_B\, Q_A$$

$$TD = Q_C\, Q_B\, Q_A$$

Synchronous 4-Bit Up Counter



Exercise - 4-bit Synchronous Up Counter Using JK FF

```verilog
module JK_FF(input J, input K, input clk, input reset, output reg Q);
    always @(posedge clk or posedge reset)
    begin
        if (reset)
            Q <= 0;
        else
            case ({J, K})
                2'b00: Q <= Q;
                2'b01: Q <= 0;
                2'b10: Q <= 1;
                2'b11: Q <= ~Q;
            endcase
    end
endmodule

```

```verilog
`include "module_jkff.v"
module sync_counter(input clk, input reset, output [3:0] Q);
    wire J0, K0, J1, K1, J2, K2, J3, K3;

    assign J0 = 1;
    assign K0 = 1;
    assign J1 = Q[0];
    assign K1 = Q[0];
    assign J2 = Q[0] & Q[1];
    assign K2 = Q[0] & Q[1];
    assign J3 = Q[0] & Q[1] & Q[2];
    assign K3 = Q[0] & Q[1] & Q[2];

    JK_FF ff0(J0, K0, clk, reset, Q[0]);
    JK_FF ff1(J1, K1, clk, reset, Q[1]);
    JK_FF ff2(J2, K2, clk, reset, Q[2]);
    JK_FF ff3(J3, K3, clk, reset, Q[3]);
endmodule
```